

Java와 JVM 전체 구조

소스 코드 한 줄이 실행되고, 메모리를 사용하고, 여러 요청을 처리하기까지

이 문서는 Java 문법서가 아니다. Java 플랫폼의 구성요소가 어떻게 연결되는지, JVM 내부에서 코드가 어떤 단계를 거쳐 실행되는지를 이해하기 위한 개념 지도다.

전체 지도

Java source (.java)

↓ javac

Bytecode (.class)

↓ ClassLoader · 검증 · 연결 · 초기화

JVM Runtime Data Areas

↓ Interpreter · JIT Compiler

Native machine code / CPU

↕

Heap · Garbage Collector · OS threads · Native libraries

핵심 **Java**는 언어이고, **JVM**은 실행 규격이며, **JDK**는 개발 도구와 실행 환경의 묶음이다.

PART I. Java 플랫폼

1. Java, JVM, JRE, JDK

Java는 클래스, 인터페이스, 제네릭, 예외, 람다 같은 문법과 타입 체계를 가진 프로그래밍 언어다. Java 소스 자체를 CPU가 직접 실행하지는 않는다.

- JVM(Java Virtual Machine): .class 바이트코드를 로드하고 실행하는 가상 머신 규격
- JDK(Java Development Kit): javac, java, javadoc, jcmd, jstack 같은 도구와 표준 라이브러리
- JRE: JVM과 실행에 필요한 라이브러리를 가리키던 배포 단위. 현대 JDK에서는 별도 제품 구분이 약해졌다.
- HotSpot: OpenJDK에서 널리 사용하는 JVM 구현체. JVM 명세 자체와 동일한 말은 아니다.

핵심 **JVM**은 규격이고 **HotSpot**은 구현이다. 다른 **JVM** 구현체도 같은 .class 계약을 실행할 수 있다.

2. 컴파일과 실행

```
javac Hello.java      # 소스 → Hello.class
```

```
java Hello            # JVM 시작 → class 로드 → main 실행
```

javac는 소스를 특정 CPU 명령어가 아니라 JVM 바이트코드로 변환한다. 바이트코드는 플랫폼 중립적인 중간 명령 집합이다. JVM 구현체가 실행 중 이를 해석하거나 현재 CPU의 기계어로 컴파일한다.

Java의 '한 번 작성하고 여러 환경에서 실행'이라는 성질은 같은 바이트코드를 각 운영체제용 JVM이 받아 실행하는 구조에서 나온다. 운영체제와 CPU에 맞는 JVM 자체는 별도로 존재한다.

3. .class 파일과 바이트코드

.class 파일에는 클래스 이름, 상위 타입, 필드와 메서드 정보, 바이트코드, 상수 풀, 애노테이션 등의 구조화된 정보가 들어 있다. 단순한 기계어 파일이 아니다.

- 바이트코드는 스택 기반 명령어다. 피연산자를 operand stack에 넣고 연산한다.
- constant pool은 문자열, 타입, 필드와 메서드 참조 등을 보관한다.
- 디버그 정보는 컴파일 옵션에 따라 줄 번호와 지역 변수 이름 등을 포함할 수 있다.

```
iconst_1      // 정수 1을 operand stack에 push

iconst_2      // 정수 2를 push

iadd          // 두 값을 꺼내 더한 뒤 결과 push
```

PART II. JVM이 클래스를 실행하기까지

4. ClassLoader Subsystem

JVM은 필요한 클래스를 처음부터 전부 읽지 않고, 실제로 필요해질 때 로드한다. 클래스 로딩은 크게 Loading, Linking, Initialization으로 나뉜다.

Loading

↓

Linking: Verification → Preparation → Resolution

↓

Initialization

- Loading: 클래스 바이트를 찾아 JVM 내부의 Class 객체로 만든다.
- Verification: 바이트코드가 JVM 형식과 타입 안전 규칙을 만족하는지 확인한다.
- Preparation: static 필드 공간을 만들고 기본값으로 준비한다.
- Resolution: constant pool의 심볼 참조를 실제 클래스·필드·메서드 참조로 연결한다.
- Initialization: static 필드 초기화식과 static 블록을 실행한다.

핵심 클래스 로딩은 파일 읽기만이 아니라 검증, 연결, 초기화까지 이어지는 생명주기다.

5. ClassLoader 계층과 위임

Bootstrap ClassLoader

↓

Platform ClassLoader

↓

Application ClassLoader

↓

WebApp / Plugin ClassLoader

일반적으로 자식 ClassLoader는 클래스를 직접 찾기 전에 부모에게 위임한다. 이 parent-first 원칙은 핵심 Java 클래스가 애플리케이션 클래스에 의해 임의로 대체되는 것을 막고 타입 정체성을 안정시킨다.

같은 이름의 클래스라도 서로 다른 ClassLoader가 정의하면 JVM에서는 서로 다른 타입이다. 따라서 플러그인이 나 애플리케이션 서버에서 ClassCastException과 클래스 누수 문제가 발생할 수 있다.

6. Thread Context ClassLoader

상위 ClassLoader가 로드한 공통 라이브러리는 하위 WebApp ClassLoader에만 존재하는 구현체를 직접 볼 수 없다. 이를 위해 스레드가 현재 실행 문맥의 ClassLoader 참조를 보관할 수 있다.

```
Thread.currentThread().getContextClassLoader()
```

이 API는 동시성 제어가 아니다. ServiceLoader, JDBC, 애플리케이션 서버, 플러그인처럼 실행 문맥에 따라 로딩 영역이 달라질 때 사용하는 간접 로딩 장치다.

PostForge처럼 하나의 Spring Boot 애플리케이션이 고정 classpath 리소스를 읽는다면 다음 표현이 의도에 더 가깝다.

```
PromptResourceLoader.class.getClassLoader()
```

PART III. JVM 메모리 구조

7. Runtime Data Areas

JVM 전체 공유

스레드별

┌ └	┌ └
Heap	PC Register
Method Area	JVM Stack
Runtime Constant Pool	Native Method Stack
┌ └	┌ └

JVM 명세는 논리적인 런타임 영역과 역할을 정의한다. 실제 메모리 배치와 내부 자료구조는 JVM 구현체가 결정한 다.

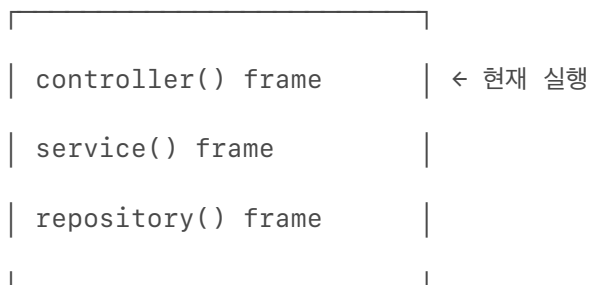
- Heap: 일반적인 객체와 배열이 위치하며 여러 스레드가 공유한다.
- Method Area: 클래스 메타데이터, 메서드 코드, static 정보 등을 위한 논리 영역이다.
- Runtime Constant Pool: 각 클래스의 constant pool을 실행 시점 형태로 보관한다.
- PC Register: 현재 스레드가 실행 중인 JVM 명령 위치를 나타낸다.
- JVM Stack: 메서드 호출마다 생성되는 stack frame을 쌓는다.
- Native Method Stack: JNI 등 네이티브 메서드 실행을 지원한다.

핵심 **Metaspace**는 **HotSpot**의 **Method Area** 구현 방식이다. JVM 명세가 **Metaspace**라는 이름을 강제하지 않는다.

8. Stack Frame과 메서드 호출

스레드가 Java 메서드를 호출하면 JVM stack에 frame이 하나 추가된다. 메서드가 정상 반환하거나 예외로 빠져 나오면 frame이 제거된다.

Thread JVM Stack



- Local Variable Array: this, 매개변수, 지역 변수
- Operand Stack: 바이트코드 연산 중간값
- Dynamic Linking: 현재 메서드가 참조하는 다른 메서드와 필드 연결
- Return 정보: 호출자 frame으로 돌아가기 위한 상태

StackOverflowError는 호출 frame이 지나치게 쌓이는 대표적인 결과다. 객체가 항상 stack에 생성된다고 단정할 수는 없으며, JIT의 escape analysis에 따라 실제 배치는 최적화될 수 있다.

9. Heap과 객체

Java 코드에서 new로 만든 객체는 개념적으로 heap에 존재하고 참조를 통해 접근한다. 여러 변수는 같은 객체를 가리킬 수 있으며, 변수에 객체 자체가 복사되는 것은 아니다.

```
Post a = new Post();
```

```
Post b = a;           // a와 b는 같은 객체를 참조
```

객체 헤더, 필드 정렬, 압축 포인터 같은 실제 object layout은 HotSpot 옵션과 JVM 구현에 따라 달라진다. Java 언어 수준의 계약과 특정 JVM의 메모리 배치를 구분해야 한다.

PART IV. 실행 엔진과 최적화

10. Interpreter와 JIT Compiler

JVM은 처음 만난 바이트코드를 interpreter로 빠르게 실행할 수 있다. 자주 실행되는 hot method와 loop를 관찰한 뒤 JIT compiler가 현재 CPU에 최적화된 기계어로 컴파일한다.

Bytecode

- └ Interpreter: 시작이 빠름
- └ JIT Compiler: 반복 실행이 빠름

↓

Code Cache의 native code

- Inlining: 작은 메서드 호출을 호출자 코드에 펼친다.
- Devirtualization: 실제 호출 타입이 안정적이면 동적 호출 비용을 줄인다.
- Escape analysis: 객체가 메서드나 스레드 밖으로 탈출하는지 분석한다.
- Deoptimization: 최적화 가정이 깨지면 안전한 실행 상태로 되돌린다.

핵심 Java 애플리케이션은 실행 중 프로파일을 보고 최적화되므로 단순한 첫 실행 시간만으로 정상 상태 성능을 판단하면 안 된다.

11. JNI와 네이티브 영역

JVM은 JNI(Java Native Interface)를 통해 C/C++ 같은 네이티브 코드와 상호작용할 수 있다. 파일 시스템, 네트워크, 압축, 암호화 같은 표준 라이브러리 내부에서도 네이티브 기능을 사용할 수 있다.

네이티브 코드는 JVM의 메모리 안전성과 GC 관리 밖에 있는 자원을 다룰 수 있으므로 크래시, 메모리 누수, 플랫폼 종속성이 생길 수 있다. 필요한 경계에서만 사용한다.

PART V. Garbage Collection

12. GC의 기준: 도달 가능성

GC는 변수 이름이나 scope 종료만으로 객체를 삭제하지 않는다. GC Root에서 참조 경로를 따라 도달할 수 있는지를 기준으로 살아 있는 객체를 판단한다.

GC Roots

- └ 실행 중인 stack의 참조
- └ static 필드의 참조
- └ JNI 참조
- └ JVM 내부 참조

↓ reachable

live objects

더 이상 어떤 GC Root에서도 도달할 수 없는 객체는 회수 후보가 된다. 언제 정확히 회수되는지는 보장되지 않는다.

13. GC 과정과 Stop-The-World

GC 구현은 살아 있는 객체 표시, 죽은 객체 회수, 메모리 압축 또는 영역 재배포 같은 작업을 조합한다. 일부 단계에서는 애플리케이션 스레드를 멈추는 Stop-The-World 구간이 필요하다.

- Throughput: 전체 시간 중 애플리케이션 작업 비율
- Latency: GC로 인해 개별 요청이 멈추는 시간
- Footprint: 목표 성능을 위해 사용하는 메모리 크기

G1, ZGC, Serial GC 같은 이름은 HotSpot의 구체적인 collector 선택지다. JVM 명세는 특정 GC 알고리즘이나 세대 구조를 강제하지 않는다.

14. 메모리 누수는 Java에도 있다

GC가 있어도 더 이상 필요하지 않은 객체를 static collection, listener, ThreadLocal, cache 등이 계속 참조하면 회수할 수 없다. 이것이 Java의 논리적 메모리 누수다.

```
static final Map<String, Object> CACHE = new HashMap<>();
```

```
// 제거 정책 없이 계속 추가하면 객체는 여전히 reachable
```

핵심 GC는 '참조되지 않는 객체'를 회수한다. '개발자가 더 이상 필요하지 않은 객체'를 판단하지 않는다.

PART VI. 스레드와 Java Memory Model

15. Java Thread와 실행

각 Java 스레드는 자신의 PC register와 JVM stack을 가지며 heap은 공유한다. 전통적인 platform thread는 일반적으로 운영체제 스레드와 대응한다. Java 21의 virtual thread는 많은 Java 작업을 적은 carrier thread 위에서 스케줄링할 수 있게 한다.

스레드별 stack이 분리되어 있어도 stack 안의 참조가 가리키는 heap 객체는 여러 스레드가 함께 사용할 수 있다. 동시성 문제는 이 공유 객체를 읽고 수정할 때 발생한다.

16. Java Memory Model(JMM)

JMM은 여러 스레드가 메모리 읽기와 쓰기를 어떤 순서로 관찰할 수 있는지 정의한다. 소스 코드 순서가 다른 스레드에서도 그대로 보인다고 가정할 수 없다. 컴파일러, JIT, CPU가 허용 범위 안에서 명령과 메모리 접근을 재배포할 수 있기 때문이다.

- synchronized: 상호 배제와 함께 lock 해제·획득 사이의 가시성을 제공한다.
- volatile: 해당 변수의 읽기·쓰기에 가시성과 순서 규칙을 제공하지만 복합 연산 전체를 원자화하지는 않는다.
- AtomicInteger 등: compare-and-set을 이용한 원자 연산을 제공한다.
- final 필드: 올바르게 생성·공개된 객체의 초기화 안전성에 특별한 규칙이 있다.

```
count++; // read → add → write의 복합 연산. volatile만으로 원자적이지 않다.
```

17. happens-before

happens-before는 실제 시계 시간이 아니라 한 작업의 결과가 다른 작업에 반드시 보이도록 만드는 JMM의 순서 관계다.

- 한 스레드 안의 앞선 작업은 뒤 작업보다 happens-before
- synchronized unlock은 같은 monitor의 이후 lock보다 happens-before
- volatile write는 같은 변수의 이후 read보다 happens-before
- Thread.start() 이전 작업은 시작된 스레드의 작업보다 happens-before
- 스레드의 작업은 다른 스레드가 성공적으로 join()한 이후보다 happens-before

18. PromptResourceLoader 캐시 사례

```
private final ConcurrentMap<String, String> cache
```

```
    = new ConcurrentHashMap<>();
```

```
return cache.computeIfAbsent(path, this::readResource);
```

MD 파일이 정적이라는 사실은 파일을 동시에 읽는 일을 위험하게 만들지 않는다. 동시성 고려가 필요한 이유는 여러 요청이 cache라는 변경 가능한 상태를 공유하기 때문이다.

캐시를 없애고 요청마다 독립적으로 파일을 읽는다면 공유 Map에 대한 동시성 제어도 사라진다. lazy cache를 유지한다면 최초 cache miss가 겹칠 수 있으므로 ConcurrentHashMap이 가장 작은 안전한 선택이다.

PART VII. Spring Boot 애플리케이션에 연결하기

19. HTTP 요청 하나가 실행되는 경로

Client

↓ TCP / HTTP

Embedded server

↓ request thread 또는 virtual thread

Spring Filter Chain

↓ DispatcherServlet

Controller → Service → Repository / External API

↓

Response serialization → HTTP response

이 모든 단계는 JVM 프로세스 안에서 실행된다. Spring bean 객체는 heap에 있고, 각 요청 스레드는 자신의

stack frame을 쌓으며 같은 singleton bean을 공유한다. singleton bean에 변경 가능한 필드를 두면 요청 간 동시성 문제가 생길 수 있다.

20. PostForge AI 채팅 흐름

ChatController

↓

ChatService

- |─ AiSafetyGuard 입력 검사
- |─ SemanticSearchService 문서 검색
- |─ PromptResourceLoader 정적 MD 로드
- |─ 검색 context 조립
- └─ TextGenerationClient → LLM

↓

AiSafetyGuard 출력 검사

chat-system.md와 safety-policy.md는 classpath 리소스다. PromptResourceLoader는 이를 문자열로 읽고 캐시한다. 검색 context는 요청마다 달라지므로 ChatService가 정적 프롬프트와 동적으로 조립한다.

21. 빌드와 배포

Gradle

- |─ compileJava: .java → .class
- |─ processResources: MD/YAML 등 복사
- |─ test: 테스트 JVM 실행
- └─ bootJar/jar: 클래스와 리소스 패키징

↓

java -jar app.jar

운영에서 MD 파일은 일반적으로 JAR 내부 classpath에 포함된다. 실행 중 소스 디렉터리의 MD를 수정해도 이미 패키징된 리소스와 메모리 캐시는 바뀌지 않는다. 변경 반영은 재빌드와 재배포를 전제로 한다.

PART VIII. 관찰과 문제 해결

22. 무엇을 관찰할 것인가

- Thread dump: 어떤 스레드가 BLOCKED, WAITING, RUNNABLE인지 확인
- Heap dump: 어떤 객체가 메모리를 점유하고 누가 참조하는지 확인
- GC log: pause 시간, 할당 속도, heap 변화 확인
- JFR(Java Flight Recorder): CPU, allocation, lock, I/O 이벤트를 낮은 오버헤드로 기록
- jcmd: 실행 중 JVM의 thread, heap, class, JFR 정보를 요청

```
jcmd <pid> Thread.print
```

```
jcmd <pid> GC.heap_info
```

```
jcmd <pid> JFR.start name=postforge duration=60s filename=postforge.jfr
```

23. 자주 섞이는 개념

- JVM stack과 자료구조 Stack은 다른 개념이다.
- static은 thread-safe를 의미하지 않는다. 오히려 여러 요청이 공유할 가능성이 높다.
- volatile은 모든 동시성 문제를 해결하지 않는다. 복합 연산은 여전히 경쟁할 수 있다.
- GC가 있다고 메모리 누수가 사라지는 것은 아니다.
- ClassLoader는 단순 파일 로더가 아니라 타입 정체성과 격리 경계를 만든다.
- JVM 명세의 Method Area와 HotSpot의 Metaspace를 완전히 같은 계층의 용어로 취급하면 안 된다.

24. 학습 순서

1. Java 언어와 객체·참조
2. javac와 bytecode
3. ClassLoader 생명주기
4. stack frame과 heap
5. interpreter와 JIT
6. GC reachability
7. Thread와 JMM
8. Spring 요청 흐름에 다시 연결

각 개념을 독립적으로 외우기보다 '현재 코드는 어느 스레드의 어느 frame에서 실행되고, 어떤 heap 객체를 공유하며, 어떤 ClassLoader가 타입과 리소스를 제공하는가'라는 질문으로 연결하면 전체 구조가 보인다.

용어 요약

- Bytecode: JVM이 실행하는 플랫폼 중립 명령
- ClassLoader: 클래스 바이트를 찾아 타입으로 정의하는 로딩 경계
- Stack Frame: 메서드 호출 한 번의 실행 상태
- Heap: 객체가 위치하는 JVM 공유 메모리 영역
- JIT: 실행 중 hot code를 native code로 컴파일하는 엔진

- GC Root: 객체 도달 가능성 탐색의 시작점
- JMM: 스레드 사이 메모리 관찰 규칙
- happens-before: 한 작업의 결과가 다른 작업에 보임을 보장하는 순서 관계

1차 자료

- The Java Virtual Machine Specification
<https://docs.oracle.com/javase/specs/jvms/se21/html/>
- The Java Language Specification
<https://docs.oracle.com/javase/specs/jls/se21/html/>
- OpenJDK HotSpot documentation
<https://openjdk.org/groups/hotspot/>
- JEP 444: Virtual Threads
<https://openjdk.org/jeps/444>